

DATA SCIENCE & MACHINE LEARNING PROJECT REPORT

Bengaluru House Price Prediction

An end-to-end predictive modeling system developed to estimate residential real estate values in Bengaluru using robust data cleaning, feature mapping, and advanced regression architectures.

Domain: Real Estate Analytics & Predictive Modeling

Frameworks Used: Python, Pandas, Scikit-Learn, Seaborn

Date: July 2, 2026

1. Executive Summary

Predicting real estate prices accurately is a primary challenge across metropolitan housing markets due to high volatility and heterogeneous structural attributes. This project builds a reliable predictive engine for residential properties in Bengaluru, India. Leveraging a rich dataset comprising 13,320 instances, an optimized pipeline was engineered through rigorous pre-processing, range parsing, missing value optimization, and systematic continuous mapping.

Six distinct algorithmic regression frameworks were deployed and analyzed: Linear Regression, Decision Tree Regressor, Support Vector Regressor (SVR), K-Neighbors Regressor, and Gradient Boosting Regressor, along with an ensemble Random Forest model. Through evaluation across multiple standard error metrics (R^2 , MAE, MSE, RMSE), the **Random Forest Regressor** proved to be the superior choice, explaining **62.21%** of target price variance with an overall Root Mean Squared Error (RMSE) of **96.48**.

2. Dataset Characterization & Initial Assessment

The underlying dataset captures structural parameters across 9 raw attributes describing various spatial and physical features of properties. Below is a foundational breakdown of the source columns and their data types:

Column Name	Data Type	Initial Unique Values	Description / Significance
area_type	object (Categorical)	4	Classification of area layout (e.g., Super built-up, Plot, Carpet)
availability	object (Categorical)	81	Possession timeline or structural readiness markers
location	object (Categorical)	1305	Geographic locality identifier within Bengaluru city
size	object (Categorical)	31	Physical layout configuration specified in BHK or Bedrooms
society	object (Categorical)	2688	Gated community / apartment complex identifier
total_sqft	object (Mixed alphanumeric)	2117	Physical dimension boundary, occasionally reported as a range or unit type
bath	float64 (Numeric)	19	Total count of internal bathroom amenities
balcony	float64 (Numeric)	4	Total count of external balcony units available
price	float64 (Numeric)	1994	Target Variable: Unit property valuation in Lakhs (INR)

Missing Value Diagnostics: Initial system inspection revealed severe sparsity in specific coordinates. Column `society` recorded a missing count of 5,502 entries (approx 41.3%), whereas `balcony` had 609 nulls, `bath` had 73 nulls, and `size` had 16 nulls. This required an intentional data optimization and cleaning strategy.

3. Data Optimization & Cleaning Pipeline

3.1 Feature Dimensionality Reduction

Due to the vast structural sparsity within the `society` parameter (41.3% null entries) and its lack of numeric variance mapping, it was explicitly removed. Similarly, the `availability` feature was dropped as it reflects administrative timelines rather than inherent real estate value markers.

```
df.drop('society', axis=1, inplace=True)
df.drop('availability', axis=1, inplace=True)
```

3.2 Median Statistical Imputation

To preserve observations without inducing biased shifts, structural nulls within numerical boundaries were handled using median values:

- bath missing cells were populated with the column median (**2.0**) and cast into discrete integers.
- balcony missing entries were mapped to the column median (**2.0**) and cast into discrete integers.

3.3 Transformation of Range Boundaries

The critical indicator `total_sqft` contained text fragments and text intervals (e.g., "1000 - 1285"). A transformation module was implemented to parse string intervals, split the bounds, calculate an object arithmetic mean, and cast unstructured values into safe floating-point values.

```
def convert_sqft(x):
    try:
        if '-' in str(x):
            nums = x.split('-')
            return (float(nums[0]) + float(nums[1])) / 2
        return float(x)
    except:
        return None
```

3.4 Normalization of Structural Layout (BHK vs Bedrooms)

The column `size` contained redundant configurations like "3 BHK", "3 Bedroom", and "1 RK". String processing stripped out text formats, grouping them into unified numeric benchmarks. These labels were then translated into ordered numeric classes using explicit integer index mapping.

4. Exploratory Data Analysis & Statistical Variance

A numerical correlation assessment was conducted using the parsed dataset variables to check the internal linearity between features and the property price target:

Features	area_type	size	total_sqft	bath	balcony	price
area_type	1.0000	-0.1278	-0.0496	-0.1020	0.0916	-0.0574
size	-0.1278	1.0000	0.3576	0.8818	0.2073	0.4096
total_sqft	-0.0496	0.3576	1.0000	0.3881	0.1542	0.5748
bath	-0.1020	0.8818	0.3881	1.0000	0.2083	0.4547
balcony	0.0916	0.2073	0.1542	0.2083	1.0000	0.1251
price	-0.0574	0.4096	0.5748	0.4547	0.1251	1.0000

Key Correlation Insight: The metric `total_sqft` exhibits a strong linear relationship with property value (**0.5748**), followed closely by `bath` (**0.4547**) and `size` (**0.4096**). Conversely, `area_type` and `balcony` show weak linear associations, which guided our model selection.

5. Model Implementation & Performance Benchmarking

The data was split into an 80:20 partition for training and evaluation. Six regression architectures were evaluated using standard criteria: Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the Coefficient of Determination (R^2 Score).

Regression Algorithm	R^2 Score	MAE (Lakhs)	MSE	RMSE (Lakhs)
Random Forest Regressor	0.6221	37.9961	9308.3457	96.4798
Gradient Boosting Regressor	0.6193	38.8619	9378.1821	96.8410
K-Neighbors Regressor (KNN)	0.6013	39.2593	9821.1399	99.1017
Decision Tree Regressor	0.5184	39.9938	11863.6927	108.9206
Linear Regression	0.5045	46.0840	12204.7527	110.4751
Support Vector Regressor (SVR)	0.2902	44.3075	17484.1639	132.2277

6. Conclusions & Strategic Recommendations

- 1. Non-Linear Model Superiority:** Non-linear, ensemble tree structures clearly outperformed linear approaches. The Random Forest model achieved the highest predictive accuracy ($R^2 = 0.6221$), followed closely by Gradient Boosting ($R^2 = 0.6193$).
- 2. Feature Impact:** Spatial constraints and core structural features (square footage, room configurations, and bathroom counts) carry the highest predictive weight, while secondary design choices like balcony numbers have minimal impact on real estate valuations.
- 3. Next Steps for Improvement:** Future iterations could incorporate geo-spatial clustering on the location column (e.g., target encoding or coordinating geolocation coordinates) and apply robust outlier detection (such as Isolation Forests) to help reduce the overall RMSE.